

MI VDISP API

1 Overview

1.1. Module description

The VDISP (virtual display) module is designed to combine multiple pieces of data in color spaces such as YUV/RGB into a full-width output, and specify a display window in the form of thumbnails for each data input. **Certain thumbnail windows support region overlap**.

A typical VDISP application scenario:

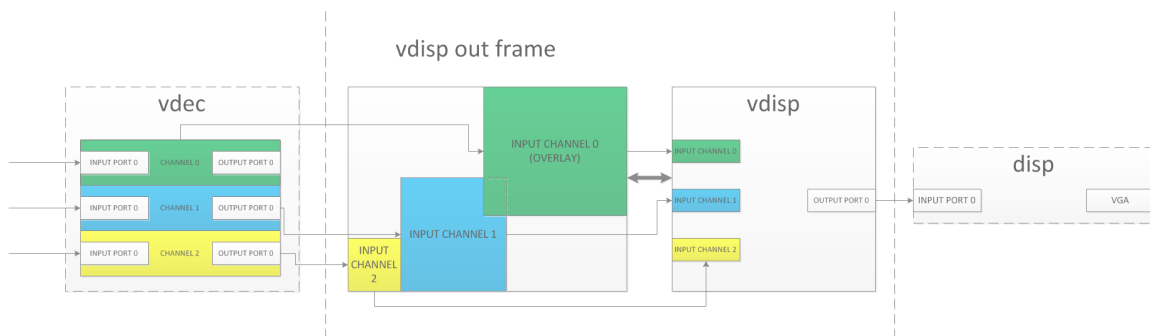


Figure 1-1 Typical application scenario of VDISP

As shown in Figure 1-1:

- vdec uses 3 channels to output 3 channels of data to VDISP to make a puzzle green channel, yellow channel and blue channel, the normal channel preview window does not support overlay, the green channel is the overlay

channel, and the overlay channel will be superimposed on all normal channels.

- vdisp receives 3 channels of vdec data, 2 channels of normal channels (yellow, blue), and one channel of overlay channel (green) to make a puzzle, and output to VGA after completion.
- disp receives the data of vdisp and outputs it with VGA signal.

1.2. Block Diagram

The special feature of VDISP is that the Input/Output Buffer is the same block. The previous module directly writes the corresponding position of the Output Buffer through Stride Write to achieve the final splicing effect. In theory, zero memory copy can be achieved.

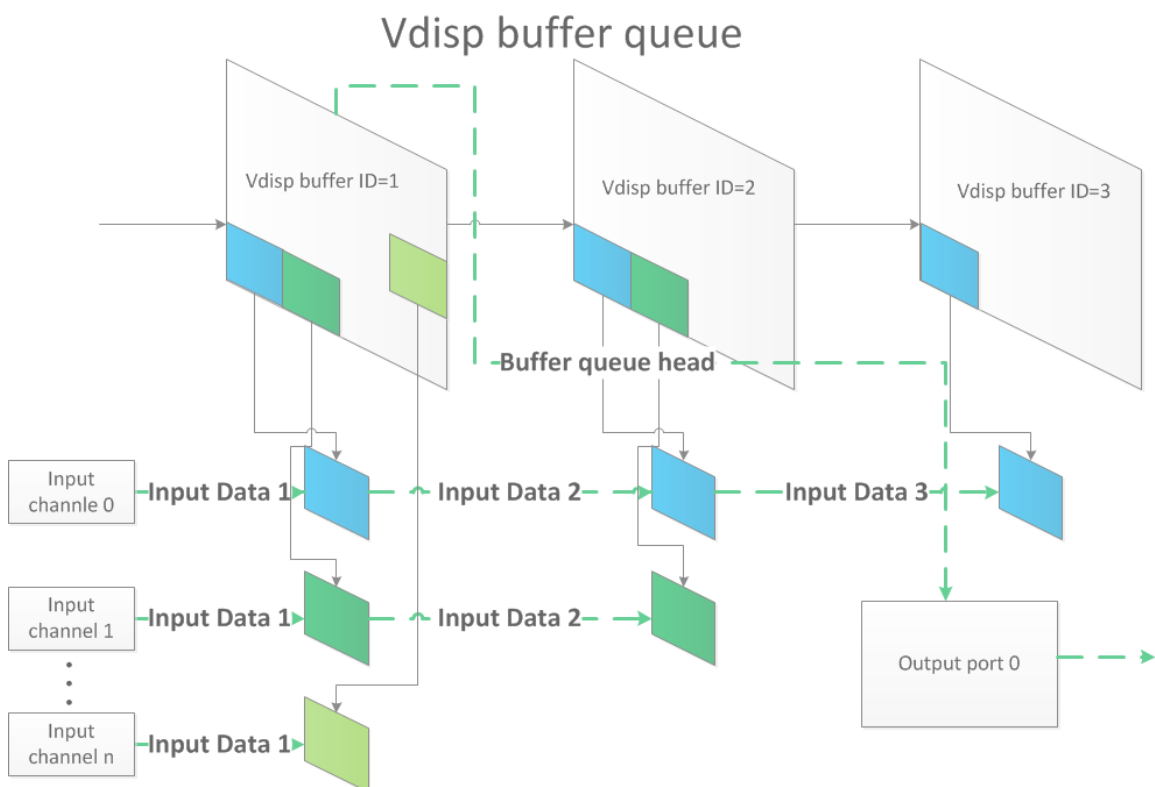


Figure 1-2 VDISP buffer queue flowchart

1.3. Constraints

1. The starting abscissa (u32OutX) of the input channel needs to be aligned, but the specific value of the alignment depends on the front-end module connected to VDISP, because hardware components read and write memory usually have address alignment requirements, otherwise it will cause anomalies such as final output lace . Usually at least 8 alignments are required, 16 is recommended.
 2. Currently 2 input data formats are supported:
 - a. E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420
 - b. E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_422
 3. 目前只支持固定帧率输出：参见[MI_VDISP_OutputPortAttr_t](#) [#35-mi_vdisp_outputportattr_t]的参数u32FrmRate
 4. VDISP 模块目前不支持：**格式转换/裁剪/**缩放****等功能。
 5. 参见更多的[规格参数](#) [#jump]。
-

2. API 参考

2.1. 功能模块API

表2-1

API名	功能
MI_VDISP_Init [#22-mi_vdisp_init]	模块初始化
MI_VDISP_Exit [#23-mi_vdisp_exit]	模块去初始化
MI_VDISP_OpenDevice [#24-mi_vdisp_opendev]	打开一个VDISP虚拟设备
MI_VDISP_CloseDevice [#25-mi_vdisp_closedev]	关闭一个VDISP虚拟设备
MI_VDISP_SetOutputPortAttr [#26-mi_vdisp_setoutputportattr]	设置输出port属性
MI_VDISP_GetOutputPortAttr [#27-mi_vdisp_getoutputportattr]	获取输出port属性
MI_VDISP_SetInputChannelAttr [#28-mi_vdisp_setinputchannelattr]	设置输入通道属性
MI_VDISP_GetInputChannelAttr [#29-mi_vdisp_getinputchannelattr]	获取输入通道属性
MI_VDISP_EnableInputChannel [#210-mi_vdisp_enableinputchannel]	使能一个输入通道
MI_VDISP_DisableInputChannel [#211-mi_vdisp_disableinputchannel]	关闭一个输入通道
MI_VDISP_StartDev [#212-mi_vdisp_startdev]	开始VDISP设备的工作
MI_VDISP_StopDev [#213-mi_vdisp_stopdev]	停止VDISP设备工作
MI_VDISP_InitDev [#214-mi_vdisp_initdev]	初始化VDISP设备
MI_VDISP_DeInitDev [#215-mi_vdisp_deinitdev]	反初始化VDISP设备

2.2. MI_VDISP_Init

- 描述

初始化vdisp模块，打开vdisp模块依赖模块，申请并初始模块内部的全局数据。

- 语法

```
MI_S32 MI_VDISP_Init (void);
```

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump1]。

- 依赖

- 头文件：mi_vdisp.h、mi_vdisp_datatype.h
- 库文件：libmi_vdisp.a(so)

- 注意

对vdisp任何操作之前,先调用本函数初始化vdisp模块

- 相关主题

[MI_VDISP_Exit](#) [#23-mi_vdisp_exit]

2.3. MI_VDISP_Exit

- 描述

退出vdisp模块，关闭vdisp模块依赖模块，析构模块内部的全局数据。

- 语法

```
MI_S32 MI_VDISP_Exit (void);
```

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump1]。

- 依赖

- 头文件：mi_vdisp.h、mi_vdisp_datatype.h
- 库文件：libmi_vdisp.a(so)
- 相关主题

[MI_VDISP_Init](#) [#22-MI_VDISP_Init]。

2.4. MI_VDISP_OpenDevice

- 描述
初始化vdisp虚拟设备，向sys注册本虚拟设备，用以和其它模块绑定。
- 语法

```
MI_S32 MI_VDISP_OpenDevice(MI_VDISP_DEV DevId)
```

- 参数

表2-2

参数名称	描述	输入/输出
DevId	本参数指定要打开的虚拟设备ID。 [0,VDISP_MAX_DEVICE_NUM), 见 规格参数 [#jump]	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump1]。
- 依赖
 - 头文件：mi_vdisp.h、mi_vdisp_datatype.h
 - 库文件：libmi_vdisp.a(so)
- 注意
调用本函数前,先使用[MI_VDISP_Init](#) [#22-mi_vdisp_init]初始化vdisp模块。

2.5. MI_VDISP_CloseDevice

- 描述

关闭vdisp虚拟设备，向sys解注册本虚拟设备，不能绑定其它模块。

- 语法

```
MI_S32 MI_VDISP_CloseDevice(MI_VDISP_DEV DevId);
```

- 参数

表2-3

参数名称	描述	输入/输出
DevId	本参数指定要关闭的虚拟设备ID。 [0,VDISP_MAX_DEVICE_NUM), 见 规格参数 [#jump]	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump1]。

- 依赖

- 头文件：mi_vdisp.h、mi_vdisp_datatype.h
- 库文件：libmi_vdisp.a(so)

- 注意

调用本函数前,先使用[MI_VDISP_StopDev](#) [#213-mi_vdisp_stopdev]停止对应的设备。

2.6. MI_VDISP_SetOutputPortAttr

- 描述

设置该vdisp虚拟设备output port的参数。

- 语法

```
MI_S32 MI_VDISP_SetOutputPortAttr(MI_VDISP_DEV DevId,  
MI_VDISP_PORT PortId,MI_VDISP_OutputPortAttr_t  
*pstOutputPortAttr);
```

- 参数

表2-4

参数名称	描述	输入/输出
DevId	目标output port所属vdisp虚拟设备的ID。 [0,VDISP_MAX_DEVICE_NUM), 见 规格参数 [#jump]	输入
PortId	目标output port的port ID。 [0,VDISP_MAX_INPUTPORT_NUM), 见 规格参数 [#jump]	输入
pstOutputPortAttr	目标outputport的配置参数指针。 MI_VDISP_OutputPortAttr_t [#35- mi_vdisp_outputportattr_t]	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump1]。

- 依赖

- 头文件：mi_vdisp.h、mi_vdisp_datatype.h
- 库文件：libmi_vdisp.a(so)

- 注意

调用本函数前,先使用[MI_VDISP_OpenDevice](#) [#24-mi_vdisp_opendevicel打开对应的vdisp设备。

2.7. MI_VDISP_GetOutputPortAttr

- 描述

获取该vdisp虚拟设备output port的参数。

• 语法

```
MI_S32 MI_VDISP_GetOutputPortAttr(MI_VDISP_DEV DevId,
MI_VDISP_PORT PortId, MI_VDISP_OutputPortAttr_t
*pstOutputPortAttr);
```

• 参数

表2-5

参数名称	描述	输入/输出
DevId	目标output port所属vdisp虚拟设备的ID。 [0,VDISP_MAX_DEVICE_NUM), 见规格参数 [#jump]	输入
PortId	目标output port的port ID。 [0,VDISP_MAX_INPUTPORT_NUM), 见规格参数 [#jump]	输入
pstOutputPortAttr	目标outputport的配置参数指针。 MI_VDISP_OutputPortAttr_t [#35- mi_vdisp_outputportattr_t]	输出

• 返回值

- 0 成功。
- 非0 失败，参照错误码 [#jump1]。

• 依赖

- 头文件：mi_vdisp.h、mi_vdisp_datatype.h
- 库文件：libmi_vdisp.a(so)

• 注意

调用本函数前,先使用MI_VDISP_OpenDevice [#24-mi_vdisp_opendevicel打开对应的vdisp设备。

2.8. MI_VDISP_SetInputChannelAttr

- 描述

设置该vdisp虚拟设备input channel的参数。

- 语法

```
MI_S32 MI_VDISP_SetInputChannelAttr(MI_VDISP_DEV DevId,  
MI_VDISP_CHN ChnId,MI_VDISP_InputChnAttr_t *pstInputChnAttr);
```

- 参数

表2-6

参数名称	描述
DevId	目标output port所属vdisp虚拟设备的ID。 [0,VDISP_MAX_DEVICE_NUM), 见规格参数 [#jump]
ChnId	目标input channel的channel ID。 [0,VDISP_MAX_CHN_NUM_PER_DEV+VDISP_MAX_OVERLAYINPUTCHN_I 见规格参数 [#jump]
pstInputChntAttr	目标input channel的配置参数指针。 MI_VDISP_InputChnAttr_t [#36-mi_vdisp_inputchnattr_t]

- 返回值

- 0 成功。
- 非0 失败，参照错误码 [#jump1]。

- 依赖

- 头文件：mi_vdisp.h、mi_vdisp_datatype.h
- 库文件：libmi_vdisp.a(so)

- 注意

调用本函数前,先使用MI_VDISP_OpenDevice [#24-mi_vdisp_opendevicel]打开对应的vdisp设备。

2.9. MI_VDISP_GetInputChannelAttr

- 描述

获取该vdisp虚拟设备input channel的参数。

- 语法

```
MI_S32 MI_VDISP_GetInputChannelAttr(MI_VDISP_DEV DevId,  
MI_VDISP_CHN ChnId,MI_VDISP_InputChnAttr_t *pstInputChnAttr);
```

- 参数

表2-7

参数名称	描述
DevId	目标output port所属vdisp虚拟设备的ID。 [0,VDISP_MAX_DEVICE_NUM), 见规格参数 [#jump]
ChnId	目标input channel的channel ID。 [0,VDISP_MAX_CHN_NUM_PER_DEV+VDISP_MAX_OVERLAYINPUTCHN_N 见规格参数 [#jump]
pstInputChnAttr	目标input channel的配置参数指针。 MI_VDISP_InputChnAttr_t [#36-mi_vdisp_inputchnattr_t]

- 返回值

- 0 成功。
- 非0 失败，参照错误码 [#jump1]。

- 依赖

- 头文件：mi_vdisp.h、mi_vdisp_datatype.h
- 库文件：libmi_vdisp.a(so)

- 注意

调用本函数前,先使用MI_VDISP_OpenDevice [#24-mi_vdisp_opendevicel]打开对应的vdisp设备。

2.10. MI_VDISP_EnableInputChannel

- 描述
使能该input channel，vdisp模块开始接收deviceId=DevId,channelId=ChnId的输入通道的数据。标记该channel状态为enable。
- 语法

```
MI_S32 MI_VDISP_EnableInputChannel(MI_VDISP_DEV DevId,  
MI_VDISP_CHN ChnId);
```

- 参数

表2-8

参数名称	描述
DevId	目标input channel所属vdisp虚拟设备的ID。 [0,VDISP_MAX_DEVICE_NUM), 见规格参数 [#jump]
ChnId	目标input channel的channel ID。 [0,VDISP_MAX_CHN_NUM_PER_DEV+VDISP_MAX_OVERLAYINPUTCHN_NUM), 见规格参数 [#jump]

- 返回值
 - 0 成功。
 - 非0 失败，参照错误码 [#jump1]。
- 依赖
 - 头文件：mi_vdisp.h、mi_vdisp_datatype.h
 - 库文件：libmi_vdisp.a(so)
- 注意
调用本函数前，先使用MI_VDISP_SetInputChannelAttr [#28-mi_vdisp_setinputchannelattr]设置对应input channel的参数。

2.11. MI_VDISP_DisableInputChannel

- 描述

用该input channel，vdisp模块停止接收deviceID=DevId,channelID=ChnId的输入通道的数据标记该channel状态为disable。

- 语法

```
MI_S32 MI_VDISP_DisableInputChannel(MI_VDISP_DEV DevId,
MI_VDISP_CHN ChnId);
```

- 参数

表2-9

参数名称	描述
DevId	目标input channel所属vdisp虚拟设备的ID。 [0,VDISP_MAX_DEVICE_NUM), 见规格参数 [#jump]
ChnId	目标input channel的channel ID。 [0,VDISP_MAX_CHN_NUM_PER_DEV+VDISP_MAX_OVERLAYINPUTCHN_NUM), 见规格参数 [#jump]



- 返回值

- 0 成功。
- 非0 失败，参照错误码 [#jump1]。

- 依赖

- 头文件：mi_vdisp.h、mi_vdisp_datatype.h
- 库文件：libmi_vdisp.a(so)

- 注意

调用本函数前,先使用MI_VDISP_SetInputChannelAttr [#28-mi_vdisp_setinputchannelattr]设置对应input channel的参数。

2.12. MI_VDISP_StartDev

- 描述

使能该虚拟设备，vdisp output port开始尝试输出input channels数据拼图之后的数据帧。使能input channel状态为enbale的所有input channel，使能output port。

- 语法

```
MI_S32 MI_VDISP_StartDev(MI_VDISP_DEV DevId);
```

- 参数

表2-10

参数名称	描述	输入/输出
DevId	本参数指定要启动的虚拟设备ID。 [0,VDISP_MAX_DEVICE_NUM), 见规格参数 [#jump]	输入

- 返回值

- 0 成功。
- 非0 失败，参照错误码 [#jump1]。

- 依赖

- 头文件：mi_vdisp.h、mi_vdisp_datatype.h
- 库文件：libmi_vdisp.a(so)

- 注意

调用本函数前,先使用MI_VDISP_OpenDevice [#24-mi_vdisp_opendevicel打开对应的vdisp虚拟设备。

2.13. MI_VDISP_StopDev

- 描述

停止该虚拟设备，vdisp output port停止输出数据帧。禁用input channel状态为enable的所有input channel以及output port，但是不修改其状态。

- 语法

```
MI_S32 MI_VDISP_StopDev(MI_VDISP_DEV DevId);
```

- 参数

表2-11

参数名称	描述	输入/输出
DevId	本参数指定要停止的虚拟设备ID。 [0,VDISP_MAX_DEVICE_NUM), 见规格参数 [jump]	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [jump1]。

- 依赖

- 头文件：mi_vdisp.h、mi_vdisp_datatype.h
- 库文件：libmi_vdisp.a(so)

- 注意

本函数会disable所有enable的输入通道/输出端口,同时清空vdisp缓存的数据帧。

2.14. MI_VDISP_InitDev

- 描述

初始化vdisp设备。

- 语法

```
MI_S32 MI_VDISP_InitDev (MI_VDISP_InitParam_t *pstInitParam);
```

- 参数

参数名称	描述	输入/输出
pstInitParam	设备初始化参数	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump1]。

- 依赖

- 头文件：mi_vdisp.h、mi_vdisp_datatype.h
- 库文件：libmi_vdisp.a(so)

- 注意

- pstInitParam暂未使用，空值传入即可。
- 此接口在Version 2.07以上版本推荐使用，用于替换原有[MI_VDISP_Init](#) [#22-mi_vdisp_init]接口。

2.15. MI_VDISP_DeInitDev

- 描述

反初始化vdisp设备。

- 语法

```
MI_S32 MI_VDISP_DeInitDev (void);
```

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump1]。

- 依赖

- 头文件：mi_vdisp.h、mi_vdisp_datatype.h

- 库文件：libmi_vdisp.a(so)
- 注意
此接口在Version 2.07以上版本推荐使用，用于替换原有MI_VDISP_Exit [#23-mi_vdisp_exit]接口。

3. 数据类型

3.1. 数据类型定义

MI_VDISP相关数据类型、数据结构定义如下表。

表3-1

数据类型	定义
MI_VDISP_DEV [#32-mi_vdisp_dev]	定义device id类型
MI_VDISP_PORT [#33-mi_vdisp_port]	定义port id类型
MI_VDISP_CHN [#34-mi_vdisp_chn]	定义channel id 类型
MI_VDISP_OutputPortAttr_t [#35-mi_vdisp_outputportattr_t]	定义输出端口属性
MI_VDISP_InputChnAttr_t [#36-mi_vdisp_inputchnattr_t]	定义输入端口属性
MI_VDISP_InitParam_t [#37-mi_vdisp_initparam_t]	定义VDISP设备初始化参数

3.2. MI_VDISP_DEV

- 说明
定义device id类型。
- 定义

```
typedef MI_S32 MI_VDISP_DEV;
```

- 注意事项

小于0为无效值

3.3. MI_VDISP_PORT

- 说明

定义port id类型。

- 定义

```
typedef MI_S32 MI_VDISP_PORT;
```

- 注意事项

小于0为无效值。

3.4. MI_VDISP_CHN

- 说明

定义channel id类型。

- 定义

```
typedef MI_S32 MI_VDISP_CHN;
```

- 注意事项

小于0为无效值。

3.5. MI_VDISP_OutputPortAttr_t

- 说明

定义输出端口属性。

- 定义

```
typedef struct MI_VDISP_OutputPortAttr_s
{
    MI_U32 u32BgColor; /* Background color of a output port, in
YUV > format.
    [23:16]:v, [15:8]:y, [7:0]:u*/
    MI_SYS_PixelFormat_e ePixelFormat; /* pixel format of a
output port */
    MI_U64 u64pts; /* current PTS */
    MI_U32 u32FrmRate; /* the frame rate of output port */
    MI_U32 u32Width; /* the frame width of a output port */
    MI_U32 u32Height; /* the frame height of a output port */
} MI_VDISP_OutputPortAttr_t;
```

- 成员

表3-2

成员名称	描述
u32BgColor	vdisp用来设置给未使用的预览窗口区域填充的颜色(yuv颜色空间)。很多情况下vdisp的整张frame并不是所有的区域都会有预览窗口被使用，这些区域需要被填充默认值，否则会输出杂讯。
ePixelFormat	输出frame的像素格式。
u64pts	vdisp输出frame的合法最早PTS，当前输入数据的PTS小于这个PTS，不会被vidsp输出。
u32FrmRate	vdisp输出frame的帧率，vdisp采用固定帧率输出，如果输入数据的帧率不足这个帧率，VDISP会使用上一次收到的frame插帧。
u32Width	vdisp输出frame的宽。
u32Height	vdisp输出frame的高。

3.6. MI_VDISP_InputChnAttr_t

- 说明

vdisp的input channel属性。

- 定义

```
typedef struct MI_VDISP_InputPortAttr_s
{
    MI_U32 u32OutX; /* the output frame X position of this
input port */
    MI_U32 u32OutY; /* the output frame Y position of this
input port */
    MI_U32 u32OutWidth; /* the output frame width of this input
port */
    MI_U32 u32OutHeight; /* the output frame height of this
input port */
    MI_S32 s32IsFreeRun; /* is this port free run */
} MI_VDISP_InputChnAttr_t;
```

- 成员

表3-3

成员名称	描述
u32OutX	vdisp输入通道在输出的frame的预览窗口的起始横坐标
u32OutY	vdisp输入通道在输出的frame的预览窗口的起始纵坐标
u32OutWidth	vdisp输入通道在输出的frame的预览窗口的宽
u32OutHeight	vdisp输入通道在输出的frame的预览窗口的高
s32IsFreeRun	vdisp输入通道frame数据的PTS是否被检查 TRUE: 不被检查，始终能够输出 FALSE: 被检查，小于output port 的pts则不被输出

3.7. MI_VDISP_InitParam_t

- 说明

VDISP设备初始化参数。

- 定义

```
typedef struct MI_DIVP_InitParam_s
{
    MI_U32 u32DevId;
    MI_U8 *u8Data;
} MI_DIVP_InitParam_t;
```

- 成员

表3-4

成员名称	描述
u32DevId	设备ID
u8Data	数据指针buffer

- 相关数据类型及接口

[MI_VDISP_InitDev](#) [#214-mi_vdisp_initdev]

4. 错误码

表4-1 API错误码

错误代码	宏定义	描述
0XA0142000	MI_SUCCESS	操作成功
0XA0142031	MI_VDISP_ERR_FAIL	vdisp 函数执行失败，参见系统log
0XA0142001	MI_VDISP_ERR_INVALID_DEVID	函数的dev参数非法
0XA0142003	MI_VDISP_ERR_ILLEGAL_PARAM	函数的某个参数非法，参见系统log
0XA0142008	MI_VDISP_ERR_NOT_SUPPORT	函数不支持当前参数设定
0XA0142022	MI_VDISP_ERR_MOD_INITED	vdisp已经初始化过
0XA0142021	MI_VDISP_ERR_MOD_NOT_INIT	vdisp还没有初始化
0XA0142240	MI_VDISP_ERR_DEV_OPENED	vdisp dev已经open过
0XA0142241	MI_VDISP_ERR_DEV_NOT_OPEN	在调用该函数前，需要先open vdisp设备
0XA0142027	MI_VDISP_ERR_DEV_NOT_STOP	在调用该函数前，需要先stop vdisp设备
0XA0142242	MI_VDISP_ERR_DEV_NOT_CLOSE	在调用该函数前，需要先close vdisp设备
0XA0142007	MI_VDISP_ERR_NOT_CONFIG	vdisp 的input/output 没有配置
0XA0142024	MI_VDISP_ERR_PORT_NOT_DISABLE	在调用该函数，需要先disable该channel/port
0XA0142243	MI_VDISP_ERR_PORT_NOT_UNBIND	参数指定port,未绑定前后端

5. 规格参数

表5-1 vdisp模块规格参数

宏定义	参数	描述
VDISP_MAX_DEVICE_NUM	4	VDISP模块支持的虚拟设备个数
VDISP_MAX_CHN_NUM_PER_DEV	16	单个虚拟设备，支持的最大非叠加输入通道的个数
VDISP_MAX_INPUTPORT_NUM	1	单个虚拟设备的单个非叠加输入通道的入口数。
VDISP_MAX_OVERLAYINPUTCHN_NUM	4	单个虚拟设备可叠加通道的个数，该参数为built-in的常数，可以增加以支持更多的可叠加窗口
VDISP_OVERLAYINPUTCHNID	VDISP_MAX_CHN_NUM_PER_DEV	可叠加输入通道的起始ID
VDISP_MAX_OUTPUTPORT_NUM	1	单个虚拟设备的最大出口数

6. 示例代码

本节示例代码实现了[模块说明](#) [#jump2]中的场景。

6.1. 示例程序流程图

绿色部分为vdisp模块相关流程。

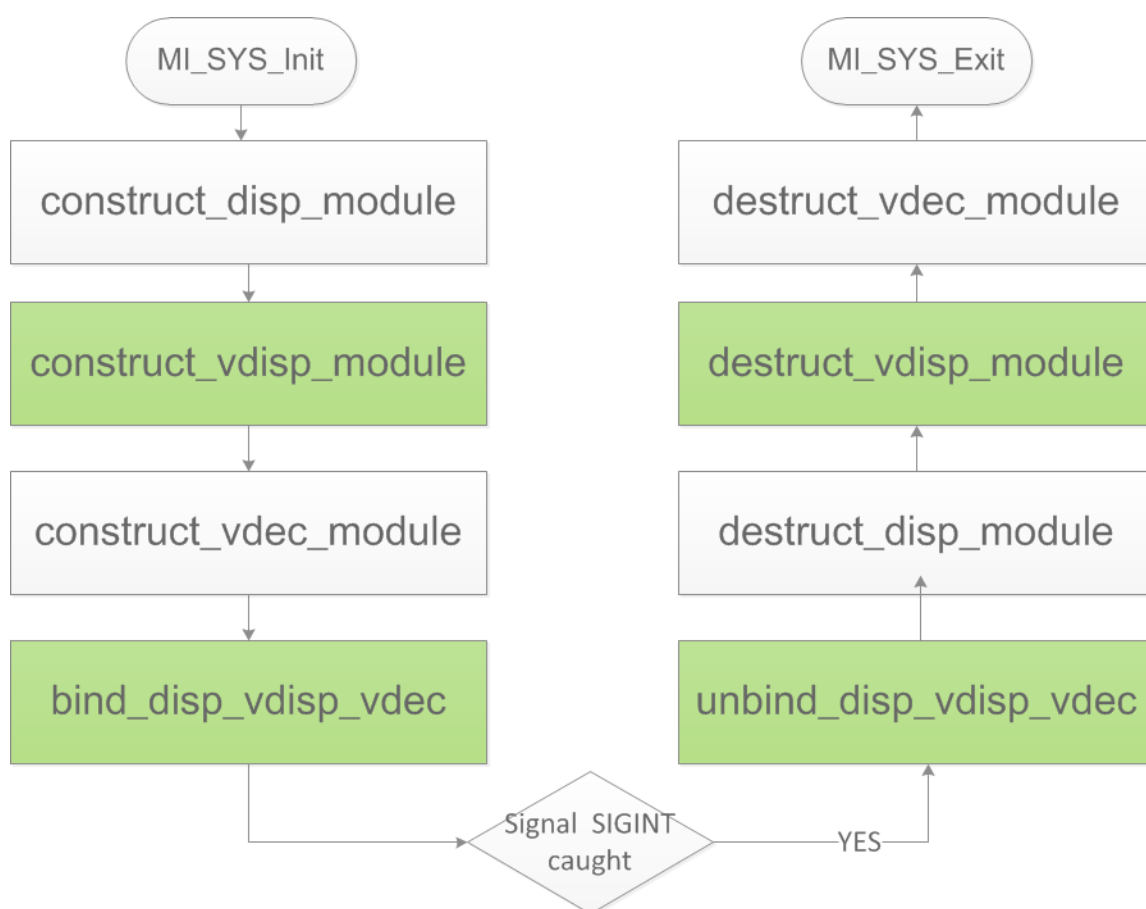


图6-1 VDISP示例流程图

6.2. vdisp相关代码

6.2.1. 构建vdisp模块

```
/*  
* 初始化vdisp模块
```



```

*      v
* 打开vdisp设备
*      v
* 设置vdisp input channel/output port 参数
*      v
* 激活input channel
*      v
* 开始vdisp设备
*/
void construct_vdisp_module(void)
{
#define MAKE_YUYV_VALUE(y,u,v)    ((y) << 24) | ((u) << 16) | ((y)
<< 8) | (v)
#define YUYV_BLACK                MAKE_YUYV_VALUE(0,128,128)
#define ALIGN16_DOWN(x) (x&0xFFF0)

    typedef struct RECT
    {
        /*
        (x,y) _ _ _ _ _
        |           |
        |           (h)
        | _ _ _ (w) _ _ _ |
        */
        short x;
        short y;
        short w;
        short h;
    } RECT;

    /*
    *初始化vdisp模块
    */
    MI_VDISP_Init();

    MI_VDISP_InputChnAttr_t stInputChnAttr;
    MI_VDISP_OutputPortAttr_t stOutputPortAttr;
    MI_VDISP_DEV DevId = 0;
    RECT rect_0, rect_1, rect_2;
    MI_VDISP_CHN Chn_0_Id = VDISP_OVERLAYINPUTCHNID;
    MI_VDISP_CHN Chn_1_Id = 1;
    MI_VDISP_CHN Chn_2_Id = 2;

    /*
    *打开一个vdisp虚拟设备,以便开始对这个设备进行配置
    */
    MI_VDISP_OpenDevice(DevId);

    /*

```

```

*为了满足vdisp缩略图窗口的对齐要求，对x坐标做16向下对齐
*设置input channel 参数
*/
rect_0.x = ALIGN16_DOWN(960);
rect_0.y = 0;
rect_0.w = 960;
rect_0.h = 960;
stInputChnAttr.s32IsFreeRun = TRUE;
stInputChnAttr.u32OutHeight = rect_0.h;
stInputChnAttr.u32OutWidth = rect_0.w;
stInputChnAttr.u32OutX = rect_0.x;
stInputChnAttr.u32OutY = rect_0.y;
/*
*为input channel VDISP_OVERLAYINPUTCHNID 设置参数
*/
MI_VDISP_SetInputChannelAttr(DevId, Chn_0_Id, &stInputChnAttr);

/*
*为了满足vdisp缩略图窗口的对齐要求，对x坐标做16向下对齐
*设置input channel 参数
*/
rect_1.x = ALIGN16_DOWN(0);
rect_1.y = 1080 - 300;
rect_1.w = ALIGN16_DOWN(300);
rect_1.h = 300;
stInputChnAttr.s32IsFreeRun = TRUE;
stInputChnAttr.u32OutHeight = rect_1.h;
stInputChnAttr.u32OutWidth = rect_1.w;
stInputChnAttr.u32OutX = rect_1.x;
stInputChnAttr.u32OutY = rect_1.y;
/*
*为input channel 1 设置参数
*/
MI_VDISP_SetInputChannelAttr(DevId, Chn_1_Id, &stInputChnAttr);

/*
*为了满足vdisp缩略图窗口的对齐要求，对x坐标做16向下对齐
*设置input channel 参数
*/
rect_2.x = ALIGN16_DOWN(300);
rect_2.y = 1080 - 700;
rect_2.w = 700;
rect_2.h = 700;
stInputChnAttr.s32IsFreeRun = TRUE;
stInputChnAttr.u32OutHeight = rect_2.h;
stInputChnAttr.u32OutWidth = rect_2.w;
stInputChnAttr.u32OutX = rect_2.x;
stInputChnAttr.u32OutY = rect_2.y;

```

```

/*
*为input channel 2 设置参数
*/
MI_VDISP_SetInputChannelAttr(DevId, Chn_2_Id, &stInputChnAttr);

//设置输出的颜色格式
stOutputPortAttr.ePixelFormat =
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420;
//设置vdisp输出帧中未使用区域的涂黑颜色,YUV 颜色空间
stOutputPortAttr.u32BgColor = YUYV_BLACK;
//设置vdisp输出帧率
stOutputPortAttr.u32FrmRate = 30;
//设置vdisp输出帧的高
stOutputPortAttr.u32Height = 1080;
//设置vdisp输出帧的宽
stOutputPortAttr.u32Width = 1920;
//设置vdisp输出帧的最低PTS
stOutputPortAttr.u64pts = 0;

/*
*为output port 0 设置参数
*/
MI_VDISP_SetOutputPortAttr(DevId,0,&stOutputPortAttr);

/*
*在设置完input channel 参数之后，
*激活对应的channel，以供使用
*/
MI_VDISP_EnableInputChannel(DevId, Chn_0_Id);
MI_VDISP_EnableInputChannel(DevId, Chn_1_Id);
MI_VDISP_EnableInputChannel(DevId, Chn_2_Id);

/*
*在vdisp 的input channel&output port 都配置完参数之后，
*让vdisp的这个设备开始工作
*/
MI_VDISP_StartDev(DevId);
}

```

6.2.2. 绑定vdisp上下游模块

```

/*
* 绑定 vdec out0 -> vdisp in0overlay---\
* 绑定 vdec out1 -> vdisp in1 ----->--> disp in0
* 绑定 vdec out2 -> vdisp in2-----/
*/
void bind_disp_vdisp_vdec(void)

```

```

{
    MI_SYS_ChnPort_t stSrcChnPort;
    MI_SYS_ChnPort_t stDstChnPort;

    /*
     * 绑定vdisp out0-> disp in0
     * <chn,dev,port> : (0,0,0) -> (0,0,0)
     */
    stSrcChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_DISP;
    stDstChnPort.u32ChnId = 0;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

    /*
     * 绑定vdec out0-> vdisp inOverlay
     * <chn,dev,port> : (0,0,0) -> (VDISP_OVERLAYINPUTCHNID,0,0)
     */
    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = VDISP_OVERLAYINPUTCHNID;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

    /*
     * 绑定vdec out0-> vdisp in1
     * <chn,dev,port> : (1,0,0) -> (1,0,0)
     */
    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
    stSrcChnPort.u32ChnId = 1;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = 1;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);
}

```

```

/*
 * 绑定vdec out2-> vdisp in2
 * <chn,dev,port> : (2,0,0) -> (2,0,0)
 */
stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
stSrcChnPort.u32ChnId = 2;
stSrcChnPort.u32DevId = 0;
stSrcChnPort.u32PortId = 0;

stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
stDstChnPort.u32ChnId = 2;
stDstChnPort.u32DevId = 0;
stDstChnPort.u32PortId = 0;
MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

}

```

6.2.3. 解绑vdisp上下游模块

与绑定过程区别很小，参见[完整代码](#) [#jump3]实现

6.2.4. 析构vdisp模块

```

/*
 * 禁用所有channel
 *      v
 * 停止vdisp设备
 *      v
 * 关闭vdisp设备
 *      v
 * 退出vdisp模块
 */
void destruct_vdisp_module(void)
{
    MI_VDISP_DEV DevId = 0;
    MI_VDISP_CHN Chn_0_Id = VDISP_OVERLAYINPUTCHNID;
    MI_VDISP_CHN Chn_1_Id = 1;
    MI_VDISP_CHN Chn_2_Id = 2;

    /*
     * 先禁用已经打开vdisp channel
     * <VDISP_OVERLAYINPUTCHNID,1,2>
     */
    MI_VDISP_DisableInputChannel(DevId, Chn_0_Id);
    MI_VDISP_DisableInputChannel(DevId, Chn_1_Id);
    MI_VDISP_DisableInputChannel(DevId, Chn_2_Id);

    /*

```

```

    *停止已经打开的vdisp设备
    */
    MI_VDISP_StopDev(DevId);
/*
    *关闭已经打开的vdisp设备
    */
    MI_VDISP_CloseDevice(DevId);
/*
    *退出vdisp模块
    */
    MI_VDISP_Exit();
}

```

6.3. 完整代码

```

#include <threads.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <assert.h>
#include <signal.h>

#include <mi_sys_datatype.h>
#include <mi_sys.h>
#include <mi_vdec_datatype.h>
#include <mi_vdec.h>
#include <mi_disp_datatype.h>
#include <mi_disp.h>
#include <mi_vdisp_datatype.h>
#include <mi_vdisp.h>

static volatile int keepRunning = 1;

static void intHandler(int dummy)
{
    keepRunning = 0;
}

void construct_disp_module(void)
{
    MI_DISP_PubAttr_t stDispPubAttr;
    stDispPubAttr.eIntfSync = E_MI_DISP_OUTPUT_1080P60;
}

```

```

    stDispPubAttr.eIntfType = E_MI_DISP_INTF_VGA;
    MI_DISP_SetPubAttr(0, &stDispPubAttr);

    MI_DISP_InputPortAttr_t stInputPortAttr;
    stInputPortAttr.u16SrcWidth = 1920;
    stInputPortAttr.u16SrcHeight = 1080;
    stInputPortAttr.stDispWin.u16X = 0;
    stInputPortAttr.stDispWin.u16Y = 0;
    stInputPortAttr.stDispWin.u16Width = 1920;
    stInputPortAttr.stDispWin.u16Height = 1080;
    MI_DISP_SetInputPortAttr(0, 0, &stInputPortAttr);

    MI_DISP_Enable(0);
    MI_DISP_EnableVideoLayer(0);
    MI_DISP_EnableInputPort(0, 0);
}

void destruct_disp_module(void)
{
    MI_DISP_DisableInputPort(0, 0);
    MI_DISP_DisableVideoLayer(0);
    MI_DISP_UnBindVideoLayer(0, 0);
    MI_DISP_Disable(0);
}

typedef struct vdecStream
{
    FILE *fp;
    MI_VDEC_CHN VdecChn;
    int frameRate;
} vdecStream;

MI_U64 get_pts(MI_U32 u32FrameRate)
{
    if(0 == u32FrameRate)
    {
        return (MI_U64)(-1);
    }

    return (MI_U64)(1000 / u32FrameRate);
}

int push_stream(void *args)
{
#define MI_U32VALUE(pu8Data, index) (pu8Data[index]<<24)|\
    (pu8Data[index+1]<<16)|(pu8Data[index+2]<<8)|(pu8Data[index+3])
#define VESFILE_READER_BATCH (128 * 1024)

    MI_S32 s32Ret = MI_SUCCESS;
    MI_VDEC_CHN VdecChn;

```

```

MI_U32 u32FrmRate = 0;
MI_U8 *pu8Buf = NULL;
MI_U32 u32Len = 0;
MI_U32 u32FrameLen = 0;
MI_U64 u64Pts = 0;
MI_U8 au8Header[16] = {0};
MI_U32 u32Pos = 0;
MI_VDEC_ChnStat_t stChnStat;
vdecStream *vdecS = (vdecStream *)args;
MI_VDEC_VideoStream_t stVdecStream;
MI_S32 s32TimeOutMs = 20;

MI_U32 u32FpBackLen = 0; // if send stream failed, file pointer
back length

VdecChn    = vdecS->VdecChn;
u32FrmRate = vdecS->frameRate;

pu8Buf = malloc(VESFILE_READER_BATCH);

while(keepRunning)
{
    ///frame mode, read one frame lenght every time

    memset(au8Header, 0, 16);
    u32Pos = fseek(vdecS->fp, 0L, SEEK_CUR);
    u32Len = fread(au8Header, 16, 1, vdecS->fp);

    if(u32Len <= 0)
    {
        fseek(vdecS->fp, 0, SEEK_SET);
        continue;
    }

    u32FrameLen = MI_U32VALUE(au8Header, 4);
    if(u32FrameLen > VESFILE_READER_BATCH)
    {
        fseek(vdecS->fp, 0, SEEK_SET);
        continue;
    }

    u32Len = fread(pu8Buf, u32FrameLen, 1, vdecS->fp);

    if(u32Len <= 0)
    {
        fseek(vdecS->fp, 0, SEEK_SET);
        continue;
    }
}

```



```

        stVdecStream.pu8Addr = pu8Buf;
        stVdecStream.u32Len = u32FrameLen;
        stVdecStream.u64PTS = u64Pts;
        stVdecStream.bEndOfFrame = 1;
        stVdecStream.bEndOfStream = 0;

        u32FpBackLen = stVdecStream.u32Len + 16; //back length

        if(MI_SUCCESS != (s32Ret = MI_VDEC_SendStream(VdecChn,
&stVdecStream, s32TimeOutMs)))
        {
            fseek(vdecS->fp, - u32FpBackLen, SEEK_CUR);
        }

        u64Pts = u64Pts + get_pts(1000 / u32FrmRate);

        memset(&stChnStat, 0x0, sizeof(stChnStat));
        MI_VDEC_GetChnStat(VdecChn, &stChnStat);
        usleep(20 * 1000);

    }

    free(pu8Buf);
    return NULL;
}

thrd_t thr0, thr1, thr2;
vdecStream vS0, vS1, vS2;

void construct_vdec_module(void)
{
    MI_VDEC_ChnAttr_t stChnAttr;
    MI_VDEC_CHN Chn_0_Id = 0;
    MI_VDEC_CHN Chn_1_Id = 1;
    MI_VDEC_CHN Chn_2_Id = 2;

    memset(&stChnAttr, 0, sizeof(MI_VDEC_ChnAttr_t));

    MI_VDEC_InitParam_t stVdecInitParam;
    MI_VDEC_ChnParam_t stChnParam;
    MI_VDEC_OutputPortAttr_t stOutputPortAttr;
    stVdecInitParam.bDisableLowLatency = FALSE;
    MI_VDEC_InitDev(&stVdecInitParam);

    stChnAttr.eCodecType      = E_MI_VDEC_CODEC_TYPE_H264;
    stChnAttr.u32PicWidth     = 1920;
    stChnAttr.u32PicHeight    = 1080;
    stChnAttr.eVideoMode      = E_MI_VDEC_VIDEO_MODE_FRAME;
    stChnAttr.u32BufSize       = 1024 * 1024;

```

```

    stChnAttr.eDpbBufMode = E_MI_VDEC_DPB_MODE_NORMAL;
    stChnAttr.stVdecVideoAttr.u32RefFrameNum = 2;
    stChnAttr.u32Priority = 0;

    MI_VDEC_CreateChn(Chn_0_Id, &stChnAttr);
    stOutputPortAttr.u16Width = 960;
    stOutputPortAttr.u16Height = 960;
    MI_VDEC_SetOutputPortAttr(Chn_0_Id, &stOutputPortAttr);

    MI_VDEC_CreateChn(Chn_1_Id, &stChnAttr);
    stOutputPortAttr.u16Width = 300;
    stOutputPortAttr.u16Height = 300;
    MI_VDEC_SetOutputPortAttr(Chn_1_Id, &stOutputPortAttr);

    MI_VDEC_CreateChn(Chn_2_Id, &stChnAttr);
    stOutputPortAttr.u16Width = 700;
    stOutputPortAttr.u16Height = 700;
    MI_VDEC_SetOutputPortAttr(Chn_2_Id, &stOutputPortAttr);

    MI_VDEC_StartChn(Chn_0_Id);
    MI_VDEC_StartChn(Chn_1_Id);
    MI_VDEC_StartChn(Chn_2_Id);

#define VDEC_FILE0 "./vdisp_demo0.h264"
#define VDEC_FILE1 "./vdisp_demo1.h264"
#define VDEC_FILE2 "./vdisp_demo2.h264"

    vS0.fp = fopen(VDEC_FILE0, "rb");
    vS0.frameRate = 30;
    vS0.VdecChn = Chn_0_Id;
    assert(vS0.fp);
    thrd_create(&thr0, push_stream, &vS0);
    vS1.fp = fopen(VDEC_FILE1, "rb");
    vS1.frameRate = 30;
    vS1.VdecChn = Chn_1_Id;
    assert(vS1.fp);
    thrd_create(&thr1, push_stream, &vS1);
    vS2.fp = fopen(VDEC_FILE2, "rb");
    vS2.frameRate = 30;
    vS2.VdecChn = Chn_2_Id;
    assert(vS2.fp);
    thrd_create(&thr2, push_stream, &vS2);
}

void destruct_vdec_module(void)
{
    thrd_join(thr0, NULL);
    thrd_join(thr1, NULL);
    thrd_join(thr2, NULL);
    MI_VDEC_StopChn(vS0.VdecChn);

```

```

        MI_VDEC_StopChn(vS1.VdecChn);
        MI_VDEC_StopChn(vS2.VdecChn);
        MI_VDEC_DestroyChn(vS0.VdecChn);
        MI_VDEC_DestroyChn(vS1.VdecChn);
        MI_VDEC_DestroyChn(vS2.VdecChn);
        MI_VDEC_DeInitDev();
    }

    /*
    * 初始化vdisp模块
    *      v
    * 打开vdisp设备
    *      v
    * 设置vdisp input channel/output port 参数
    *      v
    * 激活input channel
    *      v
    * 开始vdisp设备
    */
void construct_vdisp_module(void)
{
#define MAKE_YUYV_VALUE(y,u,v)    ((y) << 24) | ((u) << 16) | ((y) << 8) | (v)
#define YUYV_BLACK                MAKE_YUYV_VALUE(0,128,128)
#define ALIGN16_DOWN(x) (x&0xFFF0)

    typedef struct RECT
    {
        /*
        (x,y) _ _ _ _ _
        |           |
        |           (h)
        | _ _ _ (w) _ _ _ |
        */
        short x;
        short y;
        short w;
        short h;
    } RECT;

    /*
    *初始化vdisp模块
    */
    MI_VDISP_Init();

    MI_VDISP_InputChnAttr_t stInputChnAttr;
    MI_VDISP_OutputPortAttr_t stOutputPortAttr;
    MI_VDISP_DEV DevId = 0;
    RECT rect_0, rect_1, rect_2;

```

```

MI_VDISP_CHN Chn_0_Id = VDISP_OVERLAYINPUTCHNID;
MI_VDISP_CHN Chn_1_Id = 1;
MI_VDISP_CHN Chn_2_Id = 2;

/*
*打开一个vdisp虚拟设备,以便开始对这个设备进行配置
*/
MI_VDISP_OpenDevice(DevId);

/*
*为了满足vdisp缩略图窗口的对齐要求,对x坐标做16向下对齐
*设置input channel 参数
*/
rect_0.x = ALIGN16_DOWN(960);
rect_0.y = 0;
rect_0.w = 960;
rect_0.h = 960;
stInputChnAttr.s32IsFreeRun = TRUE;
stInputChnAttr.u32OutHeight = rect_0.h;
stInputChnAttr.u32OutWidth = rect_0.w;
stInputChnAttr.u32OutX = rect_0.x;
stInputChnAttr.u32OutY = rect_0.y;
/*
*为input channel VDISP_OVERLAYINPUTCHNID 设置参数
*/
MI_VDISP_SetInputChannelAttr(DevId, Chn_0_Id, &stInputChnAttr);

/*
*为了满足vdisp缩略图窗口的对齐要求,对x坐标做16向下对齐
*设置input channel 参数
*/
rect_1.x = ALIGN16_DOWN(0);
rect_1.y = 1080 - 300;
rect_1.w = ALIGN16_DOWN(300);
rect_1.h = 300;
stInputChnAttr.s32IsFreeRun = TRUE;
stInputChnAttr.u32OutHeight = rect_1.h;
stInputChnAttr.u32OutWidth = rect_1.w;
stInputChnAttr.u32OutX = rect_1.x;
stInputChnAttr.u32OutY = rect_1.y;
/*
*为input channel 1 设置参数
*/
MI_VDISP_SetInputChannelAttr(DevId, Chn_1_Id, &stInputChnAttr);

/*
*为了满足vdisp缩略图窗口的对齐要求,对x坐标做16向下对齐
*设置input channel 参数
*/

```

```

rect_2.x = ALIGN16_DOWN(300);
rect_2.y = 1080 - 700;
rect_2.w = 700;
rect_2.h = 700;
stInputChnAttr.s32IsFreeRun = TRUE;
stInputChnAttr.u32OutHeight = rect_2.h;
stInputChnAttr.u32OutWidth = rect_2.w;
stInputChnAttr.u32OutX = rect_2.x;
stInputChnAttr.u32OutY = rect_2.y;

/*
*为input channel 2 设置参数
*/
MI_VDISP_SetInputChannelAttr(DevId, Chn_2_Id, &stInputChnAttr);

//设置输出的颜色格式
stOutputPortAttr.ePixelFormat =
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420;
//设置vdisp输出帧中未使用区域的涂黑颜色,YUV 颜色空间
stOutputPortAttr.u32BgColor = YUYV_BLACK;
//设置vdisp输出帧率
stOutputPortAttr.u32FrmRate = 30;
//设置vdisp输出帧的高
stOutputPortAttr.u32Height = 1080;
//设置vdisp输出帧的宽
stOutputPortAttr.u32Width = 1920;
//设置vdisp输出帧的最低PTS
stOutputPortAttr.u64pts = 0;

/*
*为output port 0 设置参数
*/
MI_VDISP_SetOutputPortAttr(DevId, 0, &stOutputPortAttr);

/*
*在设置完input channel 参数之后，
*激活对应的channel，以供使用
*/
MI_VDISP_EnableInputChannel(DevId, Chn_0_Id);
MI_VDISP_EnableInputChannel(DevId, Chn_1_Id);
MI_VDISP_EnableInputChannel(DevId, Chn_2_Id);

/*
*在vdisp 的input channel&output port 都配置完参数之后，
*让vdisp的这个设备开始工作
*/
MI_VDISP_StartDev(DevId);
}

```

```

/*
 * 禁用所有channel
 *      v
 * 停止vdisp设备
 *      v
 * 关闭vdisp设备
 *      v
 * 退出vdisp模块
 */
void destruct_vdisp_module(void)
{
    MI_VDISP_DEV DevId = 0;
    MI_VDISP_CHN Chn_0_Id = VDISP_OVERLAYINPUTCHNID;
    MI_VDISP_CHN Chn_1_Id = 1;
    MI_VDISP_CHN Chn_2_Id = 2;

    /*
     * 先禁用已经打开vdisp channel
     * <VDISP_OVERLAYINPUTCHNID,1,2>
     */
    MI_VDISP_DisableInputChannel(DevId, Chn_0_Id);
    MI_VDISP_DisableInputChannel(DevId, Chn_1_Id);
    MI_VDISP_DisableInputChannel(DevId, Chn_2_Id);

    /*
     * 停止已经打开的vdisp设备
     */
    MI_VDISP_StopDev(DevId);

    /*
     * 关闭已经打开的vdisp设备
     */
    MI_VDISP_CloseDevice(DevId);

    /*
     * 退出vdisp模块
     */
    MI_VDISP_Exit();
}

/*
 * 绑定 vdec out0 -> vdisp inOverlay---\
 * 绑定 vdec out1 -> vdisp in1 ----->---> disp in0
 * 绑定 vdec out2 -> vdisp in2-----/
 */
void bind_disp_vdisp_vdec(void)
{
    MI_SYS_ChnnPort_t stSrcChnnPort;
    MI_SYS_ChnnPort_t stDstChnnPort;

    /*
     * 绑定vdisp out0-> disp in0

```

```

* <chn,dev,port> : (0,0,0) -> (0,0,0)
*/
stSrcChnPort.eModId = E_MI_MODULE_ID_VDISP;
stSrcChnPort.u32ChnId = 0;
stSrcChnPort.u32DevId = 0;
stSrcChnPort.u32PortId = 0;

stDstChnPort.eModId = E_MI_MODULE_ID_DISP;
stDstChnPort.u32ChnId = 0;
stDstChnPort.u32DevId = 0;
stDstChnPort.u32PortId = 0;
MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

/*
* 绑定vdec out0-> vdisp inOverlay
* <chn,dev,port> : (0,0,0) -> (VDISP_OVERLAYINPUTCHNID,0,0)
*/
stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
stSrcChnPort.u32ChnId = 0;
stSrcChnPort.u32DevId = 0;
stSrcChnPort.u32PortId = 0;

stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
stDstChnPort.u32ChnId = VDISP_OVERLAYINPUTCHNID;
stDstChnPort.u32DevId = 0;
stDstChnPort.u32PortId = 0;
MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

/*
* 绑定vdec out0-> vdisp in1
* <chn,dev,port> : (1,0,0) -> (1,0,0)
*/
stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
stSrcChnPort.u32ChnId = 1;
stSrcChnPort.u32DevId = 0;
stSrcChnPort.u32PortId = 0;

stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
stDstChnPort.u32ChnId = 1;
stDstChnPort.u32DevId = 0;
stDstChnPort.u32PortId = 0;
MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

/*
* 绑定vdec out2-> vdisp in2
* <chn,dev,port> : (2,0,0) -> (2,0,0)
*/
stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
stSrcChnPort.u32ChnId = 2;

```

```

    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = 2;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

}

void unbind_disp_vdisp_vdec(void)
{
    MI_SYS_ChnPort_t stSrcChnPort;
    MI_SYS_ChnPort_t stDstChnPort;

    stSrcChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_DISP;
    stDstChnPort.u32ChnId = 0;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_UnBindChnPort(&stSrcChnPort, &stDstChnPort);

    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = VDISP_OVERLAYINPUTCHNID;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_UnBindChnPort(&stSrcChnPort, &stDstChnPort);

    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
    stSrcChnPort.u32ChnId = 1;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = 1;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_UnBindChnPort(&stSrcChnPort, &stDstChnPort);
}

```



```

    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
    stSrcChnPort.u32ChnId = 2;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = 2;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_UnBindChnPort(&stSrcChnPort, &stDstChnPort);

}

int main(void)
{
    signal(SIGINT, intHandler);

    MI_SYS_Init();
    construct_disp_module();
    construct_vdisp_module();
    construct_vdec_module();
    bind_disp_vdisp_vdec();
    while(keepRunning)
    {
        sleep(1);
    }
    unbind_disp_vdisp_vdec();
    destruct_disp_module();
    destruct_vdisp_module();
    destruct_vdec_module();
    MI_SYS_Exit();
    return 0;
}

```

7. PROCFS介绍

7.1. cat

- 调试信息

```
# cat proc/mi_modules/mi_vdisp/mi_vdisp0
```

```
=====Private Vdisp0 Info
=====
DevStatus
  Start
----- Input
Port Info -----
PortID  PortStatus  ChnX  ChnY  ChnW  ChnH  IsFreeRun  TryOk
RecvOk
   0   Enabled    0    0    960   540    1
245356299  313332
   1   Enabled    960   0    960   540    1
355670297  313332
   2   Enabled    0   540   960   540    1
578760014  313331
   3   Enabled    960   540   960   540    1
757579130  313330
----- Output
Port Info -----
Inited  FrmInterval  BgColor  PixelFmt  FrmRate  Width  Height
SendOk
   1    33333      8388736    0        30      1920
1080    471418
```

- 调试信息分析
- 记录当前VDISP的使用状况以及device属性、Input port属性、Output port属性，可以动态地获取到这些信息，方便调试和测试。
- 参数说明

参数		描述
device info	DevStatus	Vdisp设备的工作状态 Uninit：未初始化 Init：初始化 Start：运行状态 Stop：停止状态
	PortID	Inport ID 取值范围：[0~16]，16为overlay通道，即PIP通道。
layer info	PortStatus	Port口状态 Uninit：inport未初始化 Init：inport已初始化 Enabled：inport已经使能 Disabled：inport已经禁用

	ChnX	输入inport的起始坐标X地址 取值范围：[0~4094]，需要根据chip的align对齐
	ChnY	输入inport的起始坐标Y地址 取值范围：[0~2159]
	ChnW	输入inport的图像宽度，需要根据chip的align对齐 取值范围：[0~4096]
	ChnH	输入inport的图像高度 取值范围：[0~4096]
	IsFreeRun	是否不做帧率控制 0：按pts控制播放 1：自由播放
	TryOk	尝试取前级绑定端口的数据次数
	RecvOk	从前级绑定端口成功拿到数据的次数
Output port info	Initd	是否有初始化Output Port 0：未初始化 1：已初始化
	FrmInterval	出帧时间间隔，单位US
	BgColor	背景色，此处打印的是10进制
	PixelFormat	色彩空间 取值范围： [0~E_MI_SYS_PIXEL_FRAME_FORMAT_MAX-1] 0-YUYV422，9-YUVSP420
	FrmRate	输出帧率
	Width	Image output width Value range: [0~4096], it needs to be aligned according to the align of the chip
	Height	Image output height Value range: [0~2160], it needs to be aligned according to the align of the chip

SendOk

Statistics on the number of frames successfully pushed
to the next level